Journal de bord

Mondes vivants · Sujet 04 ÉPHÉMÈRE · Simulation multi-agents du lancement d'une crème solaire fictive

> **À quoi sert ce document.** Après chaque étape du développement, on y explique ce qui a été fait, pourquoi, et ce qu'il faut retenir pour pouvoir le présenter au jury. Il se lit dans l'ordre : chaque étape s'appuie sur les précédentes. Les règles permanentes du projet sont dans `CLAUDE.md`, les décisions de conception dans `docs/contexte-projet.md`.

Le plan de travail

Le projet repart de zéro le 16 octobre 2026. Chaque étape est validée par le binôme avant de passer à la suivante, puis fait l'objet d'un commit. Les codes entre parenthèses renvoient aux exigences de l'énoncé.

#	Étape	Contenu	État
Phase 0 — Faire place nette			
0	Nettoyage du dépôt	Retrait du prototype non conforme, mise à jour de CLAUDE.md, README et journal	terminée
Phase 1 — Fondations			
1	Squelette du dépôt	Arborescence, `.gitignore`, `.env.example`, Python 3.12, ruff, fichier de configuration	terminée
2	Infrastructure Docker	Kafka (KRaft) et Redis en versions figées, création des 7 topics `sim.*`	à venir
3	Mesure de la machine	Réglages Ollama, benchmark par modèle en séquentiel puis en parallèle (E4)	à venir
Phase 2 — Le cœur, sans contenu du monde			
4	Contrat d'événements	Enveloppe Pydantic versionnée, deux horloges, `caused_by`, file `sim.dlq` (F4, C3, F6)	à venir
5	Horloge et orchestrateur	Mode synchrone, délai de livraison, `run_id` et graine (C2, C3)	à venir
6	Couche LLM	Sortie JSON imposée, routeur par palier, relance puis repli, incidents, coûts (A4, A5, E4)	à venir
7	Tranche verticale	3 agents LLM et 1 agent à règles sur 5 pas, lus par un client console	à venir
8	Historique et rejeu	Écriture Parquet, relecture d'un run sans appel LLM (E3, F5)	à venir
Phase 3 — Visualisation			
9	Pont FastAPI	WebSocket en direct ou en rejeu, inspection (D2), injection (D3)	à venir
10	Scène Godot 3D	Bâtiments distincts par zone, plage avec sable et mer, personnages animés, panneau d'indicateurs (D1)	à venir
Phase 4 — Le monde et ses agents			
11	Le monde	Nom de marque et test de contamination, population complète, seuils d'adoption	à venir

12	Corpus et RAG	Documents fictifs, ChromaDB, filtres par date et par rôle, mémoires, évaluation (F1, F2, A1, C6)	à venir
13	Agents à règles et baseline	Météo, vendeurs, concurrent, fidèles ; baseline C0 avec la même saisonnalité (A6, E1)	à venir
14	Indicateurs	Sens partagé, adoption, traçabilité, run témoin, taux de sortie de rôle (B1-B3, F6)	à venir
15	Mode asynchrone	Deuxième mode d'exécution, violations temporelles comptées (C4, C6)	à venir
Phase 5 — Preuves et soutenance			
16	Expériences	Configurations C0 à C3, au moins 5 runs chacune, Bass, synchrone/asynchrone, coûts (E1, E2, E4, E5, C5)	à venir
17	Charge et pannes	Agent muet, pic d'événements, latence jusqu'aux indicateurs (O4, C7)	à venir
18	Soutenance	Configuration de démo réduite, deux tailles de population (C8), README, run enregistré, rapport	à venir

Pourquoi cet ordre. On construit d'abord la tuyauterie (événements, horloge, appels LLM) avec très peu d'agents, pour vérifier que tout circule. Le rejeu arrive tôt (étape 8) parce qu'il permet ensuite de développer la scène 3D sur des runs enregistrés, sans occuper la carte graphique avec les modèles. Le contenu du monde (marque, corpus, population complète) vient une fois la chaîne fiable.

Étape 0 — Nettoyage du dépôt

16 octobre 2026 · Exigences concernées : aucune directement ; prépare toutes les suivantes.

Objectif

Repartir d'une base propre : ne garder dans le dépôt que ce qui est conforme aux règles du projet (`CLAUDE.md`), aux décisions de conception (`docs/contexte-projet.md`) et à l'énoncé.

Pourquoi on a repris de zéro

Un premier prototype avait été écrit avant que la conception soit arrêtée. En le comparant aux règles du projet, plusieurs écarts sont apparus, dont certains touchent des exigences notées :

Écart dans le prototype	Règle ou exigence concernée
Nom de marque, ingrédient et indice de protection écrits en dur dans le code et les prompts	Le nom doit rester un paramètre de configuration, choisi après un test de contamination
Topics `simulation.*` au lieu des 7 topics `sim.*` prévus	C1, architecture
Événements sans `run_id`, sans `caused_by`, et date de traitement en temps réel au lieu du temps simulé	C3 (deux horloges), F6 (traçabilité), E3 (rejeu)
Pas de Redis ; images Docker en version `latest`	F3 ; livrable reproductible sur une autre machine
Pas de `.gitignore`, fichiers compilés Python versionnés, arborescence différente	Conventions du dépôt
Visualisation 2D à base d'icônes	Remplacée par une scène 3D (décision du binôme)

Corriger ces écarts un par un aurait coûté plus cher que de reconstruire proprement, étape par étape.

Ce qui a été fait

- **Retrait du prototype du dépôt** : `src/`, `godot_client/`, `docker-compose.yml`, `requirements.txt`. Ces fichiers avaient été commités, ils restent donc consultables dans l'historique git (commit `607297b`).
- **Archivage hors du dépôt** des fichiers jamais commités, qu'une suppression aurait perdus définitivement : `src/agents/`, `src/rag/`, `src/simulation_engine.py`, `data/knowledge/`, les caches Godot et l'environnement virtuel. Ils sont dans `C:\Users\Siradiou\archive_prototype_S9_2026-10-16\`.
- **Mise à jour de `CLAUDE.md`** : chemin correct de l'énoncé, Python 3.12, Godot en 3D low-poly, et une section « Méthode de travail » qui fixe la règle d'avancement étape par étape.
- **Journal des décisions** complété dans `docs/contexte-projet.md`.
- **Création du `README.md`** : description du projet pour GitHub et tableau d'avancement, mis à jour à chaque commit.
- **Installation de Python 3.12.10** avec le gestionnaire d'installation officiel (`py install 3.12`), pour l'utilisateur seulement, sans droits administrateur. Python 3.14 reste installé et reste la version par défaut ; on appelle 3.12 explicitement avec `py -V:3.12`.
- **Création de ce journal de bord** : source HTML dans `docs/journal/journal-de-bord.html`, PDF généré à la racine par `docs/journal/build.ps1` (impression via Microsoft Edge, déjà présent sous Windows, donc aucune dépendance à installer).

Décisions prises à cette étape

Décision	Raison
Python **3.12**	Version stable bien prise en charge par les bibliothèques prévues (ChromaDB notamment), qui suivent souvent les nouvelles versions de Python avec retard. Python 3.12.10 a été installé à côté de 3.14 (voir ci-dessous).
Scène Godot en **3D low-poly**, rendu léger	Plus lisible et plus parlante pour le jury, tout en laissant de la mémoire graphique aux modèles LLM (budget d'environ 1 à 1,5 Go pour Windows et Godot).
Modèles 3D gratuits choisis sur maquettes	Plusieurs essais seront présentés au binôme, qui choisira (étape 10).
Avancement étape par étape, commit après validation	Chaque étape est comprise et expliquable par les deux membres avant d'aller plus loin.

État du dépôt après l'étape

```
CLAUDE.md                règles permanentes du projet
README.md                présentation et état d'avancement
journal-de-bord.pdf      ce document
docs/
  contexte-projet.md     décisions de conception
  Architecture_…pdf      schémas d'architecture
  Mondes-Vivants_…html   énoncé
  journal/
    journal-de-bord.html source de ce document
    build.ps1            génération du PDF
```

À retenir pour l'oral

- Le projet a été reconstruit sur une conception écrite et validée avant de coder, pour que chaque composant réponde à une exigence identifiée de l'énoncé.
- Interpréteur et environnement virtuel sont deux choses distinctes : Python 3.12 est installé une fois sur la machine, puis l'environnement virtuel `.venv` du projet (étape 1) est créé à partir de lui et contient toutes les bibliothèques. Ainsi les dépendances du projet n'interfèrent pas avec le reste de la machine, et la procédure se reproduit sur la machine de la collaboratrice.
- Rien n'a été perdu : l'ancien code est dans l'historique git ou archivé.

Prochaine étape

Étape 1 — Squelette du dépôt : environnement virtuel `.venv` en Python 3.12, arborescence, `.gitignore`, `.env.example`, outillage ruff, et fichier de configuration qui portera `BRAND_NAME`, le nombre d'agents, le mode d'exécution et la graine.

Étape 1 — Squelette du dépôt

16 octobre 2026 · Exigences concernées : C2 (nombre d'agents fixé au lancement), livrable « reproductible sur une autre machine, versions figées ».

Objectif

Poser le cadre dans lequel tout le code viendra se ranger : un environnement Python isolé et reproductible, une arborescence qui suit l'architecture, des outils de qualité, et un fichier de configuration qui centralise tous les paramètres d'un run.

Ce qui a été fait

- **Environnement virtuel** `.venv` créé à partir de Python 3.12.10. Toutes les bibliothèques du projet y sont installées, rien sur le reste de la machine.
- **Arborescence** : un dossier par composant de l'architecture (`sim`, `agents`, `llm`, `bus`, `rag`, `indicators`, `bridge`, `replay`), plus `corpus`, `godot`, `configs` et `tests`. Chaque paquet Python porte une phrase qui dit à quoi il sert.
- `pyproject.toml` : fiche d'identité du projet. Il déclare les dépendances directes avec leur version exacte, configure ruff (style et erreurs courantes) et pytest.
- `requirements.lock` : liste complète et figée de toutes les bibliothèques installées, y compris les dépendances indirectes.
- `.gitignore` : exclut l'environnement virtuel, les caches, le fichier de secrets `.env`, les sorties de runs et le cache de Godot.
- `.env.example` : modèle du fichier de secrets, sans valeur (adresses d'Ollama, Kafka, Redis ; clés d'API facultatives).
- **Configuration** : `configs/default.yaml` et son chargeur `sim/config.py`, plus la liste des 11 rôles dans `agents/roles.py`.
- **11 tests automatisés** dans `tests/test_config.py`.

Décisions prises à cette étape

Décision	Raison
pip + fichier verrou	Outils standards de Python, rien à apprendre en plus. Le verrou fige aussi les dépendances indirectes : la collaboratrice obtient exactement les mêmes versions.
Configuration en YAML, validée par Pydantic	Lisible et commentable, prévu dans le document d'architecture. La validation transforme une faute de frappe en erreur claire au lancement plutôt qu'en comportement faux au milieu d'un run.
Une dépendance n'arrive qu'avec l'étape qui l'utilise	Chaque bibliothèque du projet est justifiée et expliquable. Ainsi `python-dotenv` (lecture de `.env`) attendra l'étape 2.
Nom de marque provisoire `MARQUE_PROVISOIRE`	Le vrai nom sera choisi à l'étape 11 après le test de contamination. Comme il n'existe que dans la configuration, le changer ne touchera aucun code.

Comment fonctionne la configuration

Un run est entièrement décrit par un fichier YAML en quatre sections :

Section	Contenu	Valeur par défaut
`run`	Graine du générateur aléatoire	42
`simulation`	Durée simulée en jours, mode d'exécution	90 jours, synchrone
`monde`	Nom de marque, ingrédient phare, indice de protection (fictifs)	valeurs provisoires
`population`	Nombre d'agents pour chacun des 11 rôles	120 agents, dont 84 LLM

Au chargement, `load_config` vérifie chaque valeur. Sont refusés : une durée nulle, une graine négative, un mode autre que `synchrone` ou `asynchrone`, un nom de marque vide, un rôle inconnu, un effectif négatif, une population vide, et toute clé mal orthographiée (par exemple `duree_jour` au lieu de `duree_jours`). L'erreur indique le fichier et le champ fautif.

> **Lien avec l'exigence C2.** Pour changer le nombre d'agents, on modifie la section `population` du fichier YAML, ou on crée un autre fichier dans `configs/`. Le code ne change pas. Un test le vérifie avec une population de 4 agents.

Les tests

Test	Ce qu'il garantit
`test_default_config_matches_design`	La configuration par défaut correspond à la conception : 120 agents, 90 jours, mode synchrone, les 11 rôles présents.
`test_agent_count_comes_from_config`	Le nombre d'agents vient bien du fichier (C2).
`test_invalid_values_are_rejected` (7 cas)	Chaque valeur invalide listée plus haut est refusée.
`test_empty_population_is_rejected`	Un run sans agent est refusé.
`test_missing_file_gives_clear_error`	Un fichier absent donne une erreur lisible.

Résultat : 11 tests réussis, aucune remarque de ruff. La procédure d'installation du README a été rejouée dans un environnement neuf pour vérifier qu'elle fonctionne telle qu'écrite.

État du dépôt après l'étape

```
pyproject.toml        identité du projet, dépendances directes, ruff, pytest
requirements.lock     toutes les versions figées
.gitignore  .env.example
configs/default.yaml  configuration principale (120 agents, 90 jours)
sim/config.py         chargement et validation de la configuration
agents/roles.py       les 11 rôles
tests/test_config.py  11 tests
sim/ agents/ llm/ bus/ rag/ indicators/ bridge/ replay/   paquets Python
corpus/ godot/        à remplir aux étapes 10 et 12
```

À retenir pour l'oral

- Un run est entièrement décrit par un fichier de configuration et une graine : c'est la base de la reproductibilité (E2, E3) et de l'exigence C2.

- La configuration est validée avant le lancement : une erreur de saisie ne peut pas fausser un run silencieusement.
- Les versions sont figées à deux niveaux (`pyproject.toml` pour les dépendances directes, `requirements.lock` pour toutes), ce qui répond à l'exigence de livrable reproductible sur une autre machine.

Prochaine étape

Étape 2 — Infrastructure Docker : Kafka en mode KRaft et Redis, versions figées, création des 7 topics `sim.*`, et vérification que Python peut y écrire et y lire.